Time Series Tutorial 1

Handing Time Series in R

```
sales <- read.csv("sales_data.csv")

# Run once - install anything missing
pkgs <- c("tidyverse", "tsibble", "feasts", "fable", "timetk",
          "modeltime", "kableExtra", "here")
```

Handing Dates and Aggregation

```
# Load and filter data
df <- read.csv("sales_data.csv")
df$date <- as.Date(df$date)
df <- subset(df, store_id == "S001")

# Aggregate by Month using format()
agg_month <- aggregate(sales ~ format(date, "%Y-%m"), data = df, FUN = sum)

# Aggregate by Week (%U for week of the year)
agg_week <- aggregate(sales ~ format(date, "%Y-%U"), data = df, FUN = mean)

head(agg_month)
```

```
  format(date, "%Y-%m")   sales
1               2021-01 3160.42
2               2021-02 3122.53
3               2021-03 3899.81
4               2021-04 4186.90
```

```
5               2021-05 4687.30
6               2021-06 4777.31
```

```r
sales_ts <- ts(agg_month$sales, start = c(2021, 1), frequency = 12)

frequency(sales_ts)
```

```
[1] 12
```

```r
start(sales_ts)
```

```
[1] 2021    1
```

```r
end(sales_ts)
```

```
[1] 2023   12
```

```r
cycle(sales_ts)
```

```
     Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
2021   1   2   3   4   5   6   7   8   9  10  11  12
2022   1   2   3   4   5   6   7   8   9  10  11  12
2023   1   2   3   4   5   6   7   8   9  10  11  12
```

```r
time(sales_ts)
```

```
          Jan      Feb      Mar      Apr      May      Jun      Jul      Aug
2021 2021.000 2021.083 2021.167 2021.250 2021.333 2021.417 2021.500 2021.583
2022 2022.000 2022.083 2022.167 2022.250 2022.333 2022.417 2022.500 2022.583
2023 2023.000 2023.083 2023.167 2023.250 2023.333 2023.417 2023.500 2023.583
          Sep      Oct      Nov      Dec
2021 2021.667 2021.750 2021.833 2021.917
2022 2022.667 2022.750 2022.833 2022.917
2023 2023.667 2023.750 2023.833 2023.917
```

```r
window(sales_ts, start = c(2022, 1), end = c(2022, 12))
```

```
         Jan     Feb     Mar     Apr     May     Jun     Jul     Aug     Sep
2022 3813.14 3702.30 4403.61 4740.49 5231.29 5167.31 5428.39 5221.75 4747.05
         Oct     Nov     Dec
2022 4504.49 4373.27 4319.78
```

```r
stats::lag(sales_ts, k = 1)
```

```
         Jan     Feb     Mar     Apr     May     Jun     Jul     Aug     Sep
2020                                                                        
2021 3122.53 3899.81 4186.90 4687.30 4777.31 4718.29 4631.21 3992.56 3996.86
2022 3702.30 4403.61 4740.49 5231.29 5167.31 5428.39 5221.75 4747.05 4504.49
2023 4095.59 5027.84 5347.16 5661.57 5913.21 5996.13 5806.09 5034.01 5172.38
         Oct     Nov     Dec
2020                 3160.42
2021 3878.24 3763.64 3813.14
2022 4373.27 4319.78 4296.96
2023 4937.70 4957.90
```

```r
diff(sales_ts)
```

```
         Jan     Feb     Mar     Apr     May     Jun     Jul     Aug     Sep
2021          -37.89  777.28  287.09  500.40   90.01  -59.02  -87.08 -638.65
2022   49.50 -110.84  701.31  336.88  490.80  -63.98  261.08 -206.64 -474.70
2023  -22.82 -201.37  932.25  319.32  314.41  251.64   82.92 -190.04 -772.08
         Oct     Nov     Dec
2021    4.30 -118.62 -114.60
2022 -242.56 -131.22  -53.49
2023  138.37 -234.68   20.20
```

```r
diff(sales_ts, lag = 12)
```

```
        Jan    Feb    Mar    Apr    May    Jun    Jul    Aug    Sep    Oct
2022 652.72 579.77 503.80 553.59 543.99 390.00 710.10 590.54 754.49 507.63
2023 483.82 393.29 624.23 606.67 430.28 745.90 567.74 584.34 286.96 667.89
        Nov    Dec
2022 495.03 556.14
2023 564.43 638.12
```

```r
is.ts(sales_ts)
```

```
[1] TRUE
```

```r
is.na(sales_ts)
```

```
 [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
[25] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

```r
na.omit(sales_ts)
```

```
         Jan     Feb     Mar     Apr     May     Jun     Jul     Aug     Sep
2021 3160.42 3122.53 3899.81 4186.90 4687.30 4777.31 4718.29 4631.21 3992.56
2022 3813.14 3702.30 4403.61 4740.49 5231.29 5167.31 5428.39 5221.75 4747.05
2023 4296.96 4095.59 5027.84 5347.16 5661.57 5913.21 5996.13 5806.09 5034.01
         Oct     Nov     Dec
2021 3996.86 3878.24 3763.64
2022 4504.49 4373.27 4319.78
2023 5172.38 4937.70 4957.90
```

Helper conversions

```r
t <- time(sales_ts)

# Extract Year and Month
years <- floor(t)
months <- round(12 * (t - years)) + 1

data.frame(Decimal = as.numeric(t), Year = years, Month = months)
```

```
    Decimal Year Month
1  2021.000 2021     1
2  2021.083 2021     2
3  2021.167 2021     3
4  2021.250 2021     4
5  2021.333 2021     5
6  2021.417 2021     6
```

```
7  2021.500 2021     7
8  2021.583 2021     8
9  2021.667 2021     9
10 2021.750 2021    10
11 2021.833 2021    11
12 2021.917 2021    12
13 2022.000 2022     1
14 2022.083 2022     2
15 2022.167 2022     3
16 2022.250 2022     4
17 2022.333 2022     5
18 2022.417 2022     6
19 2022.500 2022     7
20 2022.583 2022     8
21 2022.667 2022     9
22 2022.750 2022    10
23 2022.833 2022    11
24 2022.917 2022    12
25 2023.000 2023     1
26 2023.083 2023     2
27 2023.167 2023     3
28 2023.250 2023     4
29 2023.333 2023     5
30 2023.417 2023     6
31 2023.500 2023     7
32 2023.583 2023     8
33 2023.667 2023     9
34 2023.750 2023    10
35 2023.833 2023    11
36 2023.917 2023    12
```

Subsetting with window

```r
# Extract only the second half of 2023
sub_ts <- window(sales_ts, start = c(2023, 7), end = c(2023, 12))
print(sub_ts)
```

```
         Jul     Aug     Sep     Oct     Nov     Dec
2023 5996.13 5806.09 5034.01 5172.38 4937.70 4957.90
```

Lags and differences

```
# Shift the time index forward by 1 period (does not move the values)
lagged_ts <- stats::lag(sales_ts, k = -1)

# First-order differencing (removes linear trend)
diff_ts <- diff(sales_ts, differences = 1)

# Seasonal differencing (removes a 12-month seasonal pattern)
seasonal_diff_ts <- diff(sales_ts, lag = 12)

diff_ts
```

```
         Jan     Feb     Mar     Apr     May     Jun     Jul     Aug     Sep
2021          -37.89  777.28  287.09  500.40   90.01  -59.02  -87.08 -638.65
2022   49.50 -110.84  701.31  336.88  490.80  -63.98  261.08 -206.64 -474.70
2023  -22.82 -201.37  932.25  319.32  314.41  251.64   82.92 -190.04 -772.08
         Oct     Nov     Dec
2021    4.30 -118.62 -114.60
2022 -242.56 -131.22  -53.49
2023  138.37 -234.68   20.20
```

Combining Series: `ts.union()` and `ts.intersect()`

```
ts_a <- ts(1:10, start = c(2023, 1), frequency = 12)
ts_b <- ts(11:18, start = c(2023, 4), frequency = 12)

# Union: keeps full range, fills gaps with NA
combined_union <- ts.union(ts_a, ts_b)

# Intersect: keeps only the overlapping months
combined_intersect <- ts.intersect(ts_a, ts_b)

combined_intersect
```

```
         ts_a ts_b
Apr 2023    4   11
May 2023    5   12
Jun 2023    6   13
```

```
Jul 2023    7   14
Aug 2023    8   15
Sep 2023    9   16
Oct 2023   10   17
```

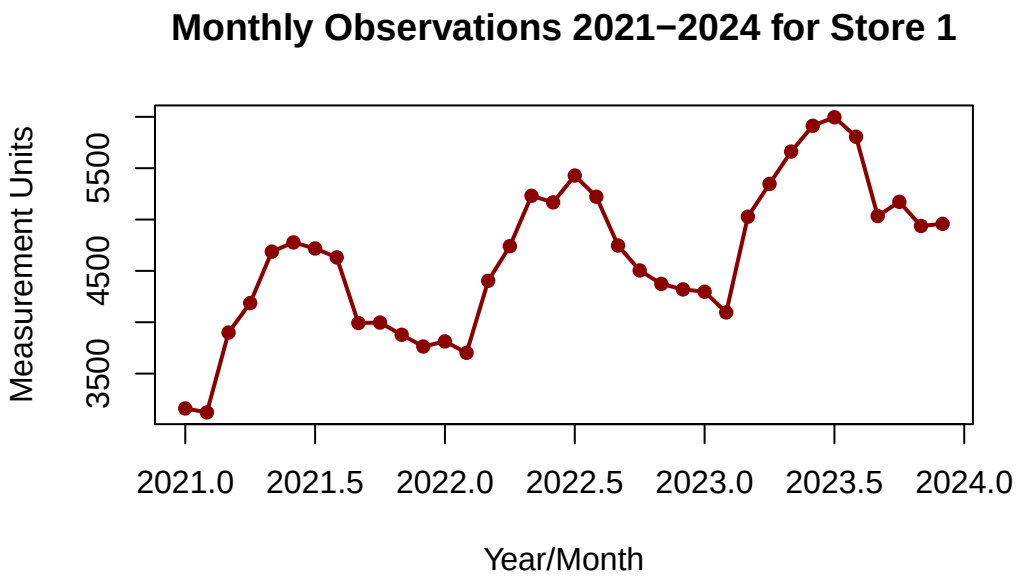
Checking for missing values

```r
has_na <- anyNA(sales_ts)
cat("Contains missing values:", has_na, "\n")
```

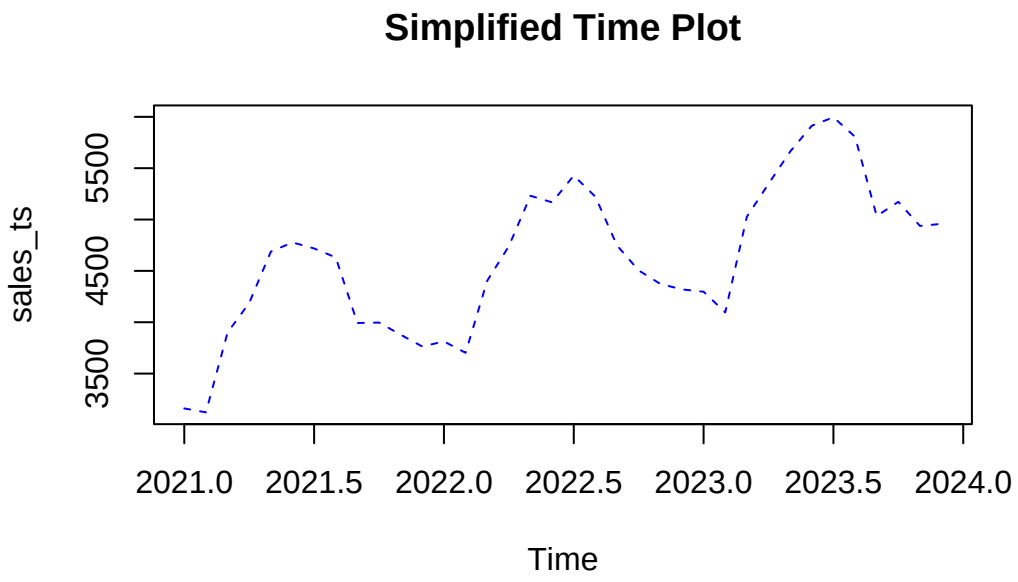
```
Contains missing values: FALSE
```

Time series plots

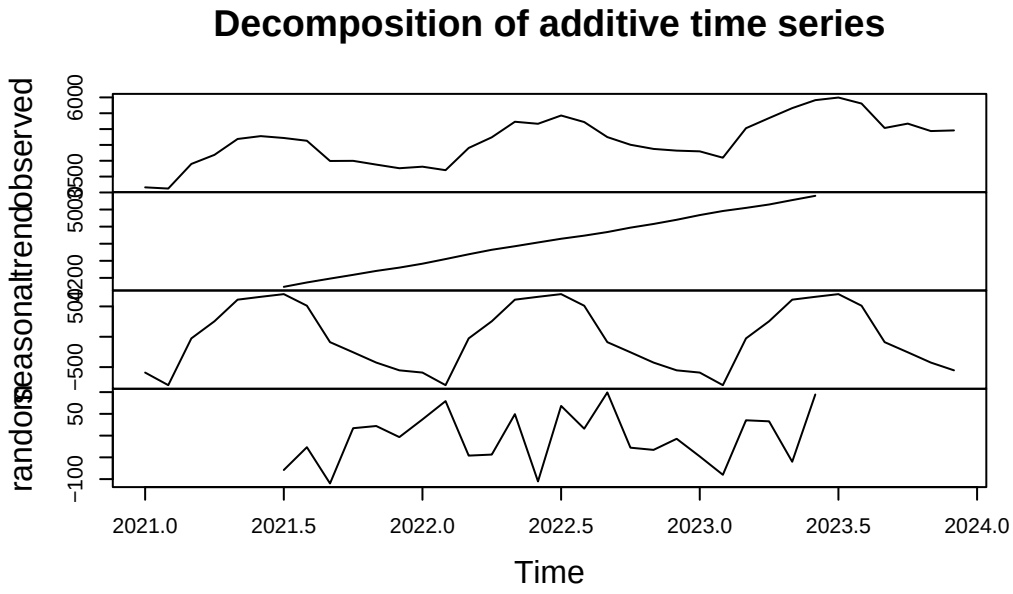
```r
plot.ts(sales_ts,
        main = "Monthly Observations 2021-2024 for Store 1",
        xlab = "Year/Month",
        ylab = "Measurement Units",
        col = "darkred",
        lwd = 2,
        lty = 1,
        type = "o",
        pch = 16)
```




```
plot(sales_ts,
     main = "Simplified Time Plot",
     col = "blue",
     lty = 2)
```




```r
# naive decomposition
decomp <- decompose(sales_ts)
plot(decomp)
```



Modern Time Series Analysis: tidyverts, timetk, and modeltime

```
# Core packages used in this section
library(tidyverse)   # dplyr, ggplot2, readr, lubridate
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.0     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.3     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.2
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```
library(tsibble)      # tidy time series class: index + key
```

```
Attaching package: 'tsibble'

The following object is masked from 'package:lubridate':

    interval

The following objects are masked from 'package:base':

    intersect, setdiff, union
```

```
library(feasts)        # Feature extraction And Statistics for Time Series
```

```
Loading required package: fabletools
```

```
library(timetk)        # tidy plotting & diagnostics for ts/tibbles
#library(modeltime)      # unified tidymodels-style forecasting framework
```

Reading Data into a `tsibble`

```r
library(readr)
library(tsibble)
library(dplyr)

# 1. Read raw data, forcing an explicit Date type on `date`
raw <- read_csv(
  "sales_data.csv",
  col_types = cols(
    date     = col_date(format = "%Y-%m-%d"),
    store_id = col_character(),
    sales    = col_double()
  )
)

# 2. Build the tsibble at its NATIVE granularity (daily) - one row per
#    (date, store_id) combination, so index and key are already unique
sales_tsbl <- raw %>%
  as_tsibble(index = date, key = store_id)

# 3. Check for implicit gaps in the time index (missing rows, not NA values)
has_gaps <- sales_tsbl %>% count_gaps()
print(has_gaps)
```

```
# A tibble: 0 x 4
# i 4 variables: store_id <chr>, .from <date>, .to <date>, .n <int>
```

```r
# 4. If gaps exist, fill them explicitly so operations don't silently misalign
sales_tsbl <- sales_tsbl %>% fill_gaps(.full = TRUE)

# 5. Only NOW aggregate to a coarser granularity, via index_by() + summarise()
sales_monthly <- sales_tsbl %>%
  index_by(month = ~ yearmonth(.)) %>%
  summarise(sales = sum(sales, na.rm = TRUE), .groups = "drop")
```

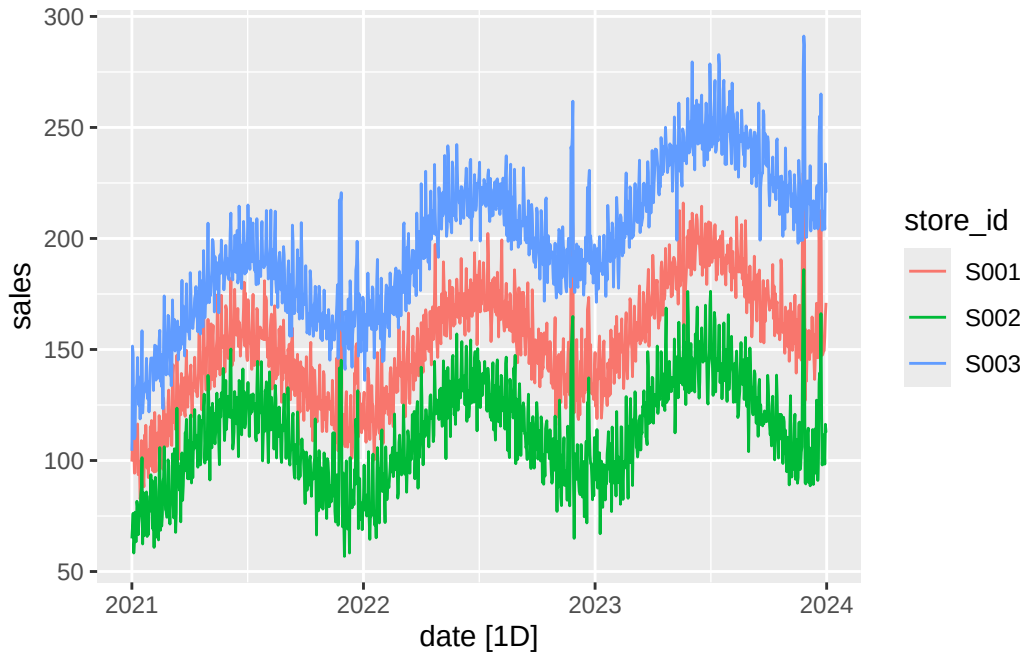
EDA with `feasts`

```r
library(feasts)
library(ggplot2)
library(tidyr)
```

```r
# Fill remaining NAs (per-store mean) before decomposition
sales_tsbl_filled <- sales_tsbl %>%
  group_by_key() %>%
  mutate(sales = tidyr::replace_na(sales, mean(sales, na.rm = TRUE))) %>%
  ungroup()

# Overall shape of the series (faceted automatically because of the key)
sales_tsbl_filled %>% autoplot(sales)
```

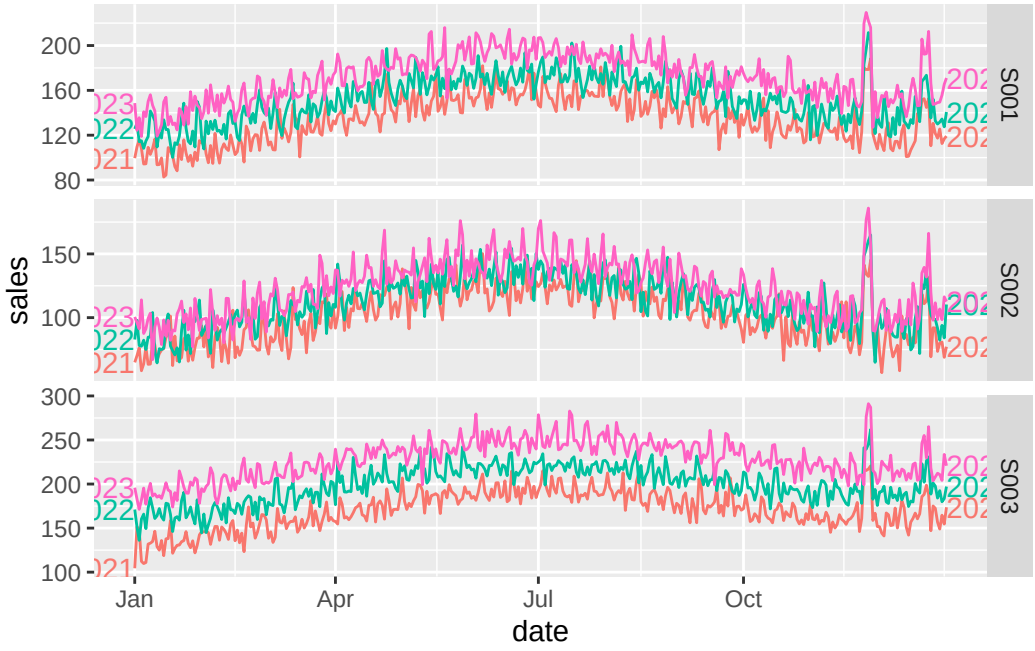
```
Warning: `autoplot.tbl_ts()` was deprecated in fabletools 0.6.0.
i Please use `ggtime::autoplot.tbl_ts()` instead.
i Graphics functions have been moved to the {ggtime} package. Please use
  `library(ggtime)` instead.
```



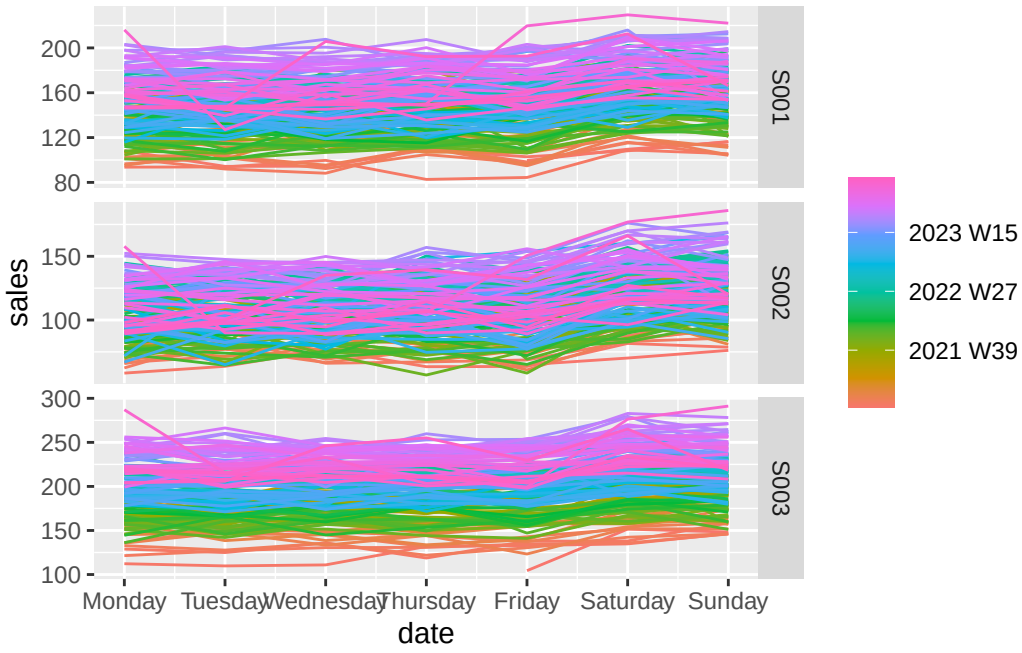

```r
# Compare seasonal cycles year over year (daily data has 2 plausible cycles -
# weekly and yearly - so `period` must be stated explicitly)
sales_tsbl_filled %>% gg_season(sales, period = "year", labels = "both")
```

```
Warning: `gg_season()` was deprecated in feasts 0.4.2.
i Please use `ggtime::gg_season()` instead.
```

```
i Graphics functions have been moved to the {ggtime} package. Please use
  `library(ggtime)` instead.
```

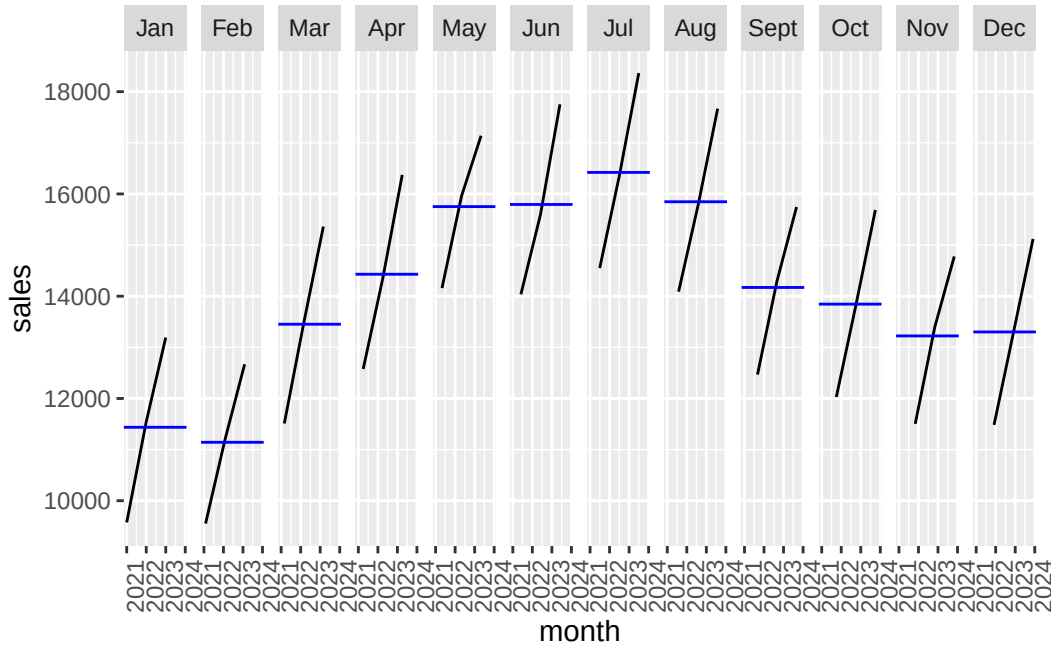



```
sales_tsbl_filled %>% gg_season(sales, period = "week")
```



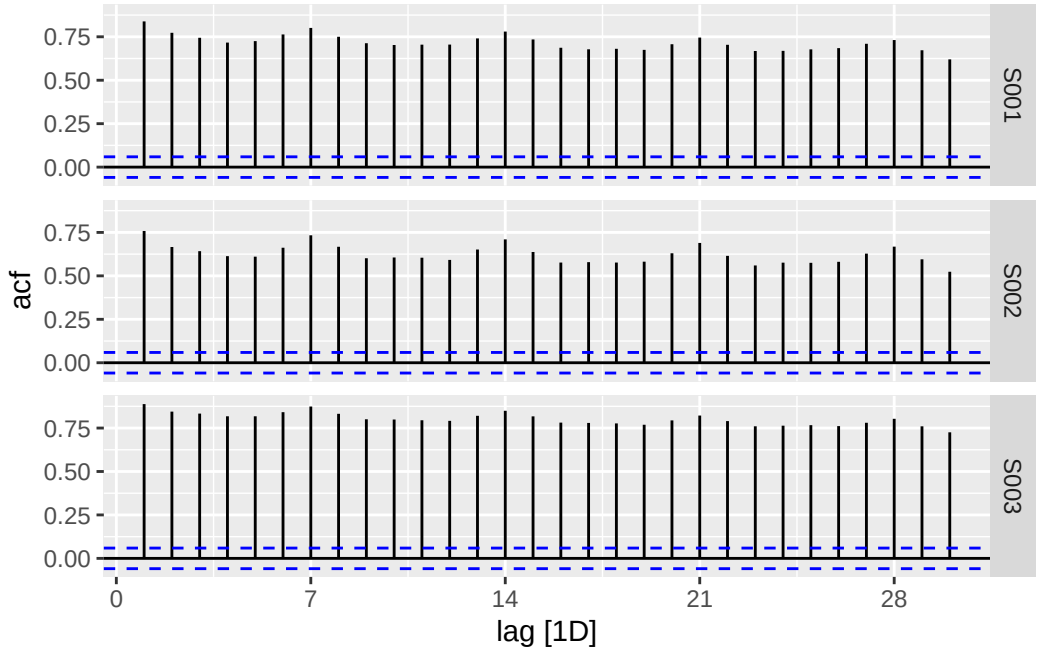

```
# Compare each calendar month's behaviour across years (the monthly aggregate
# is cleaner here than daily, since it removes the weekly cycle from view)
sales_monthly %>% gg_subseries(sales)
```

```
Warning: `gg_subseries()` was deprecated in feasts 0.4.2.
i Please use `ggtime::gg_subseries()` instead.
i Graphics functions have been moved to the {ggtime} package. Please use
  `library(ggtime)` instead.
```



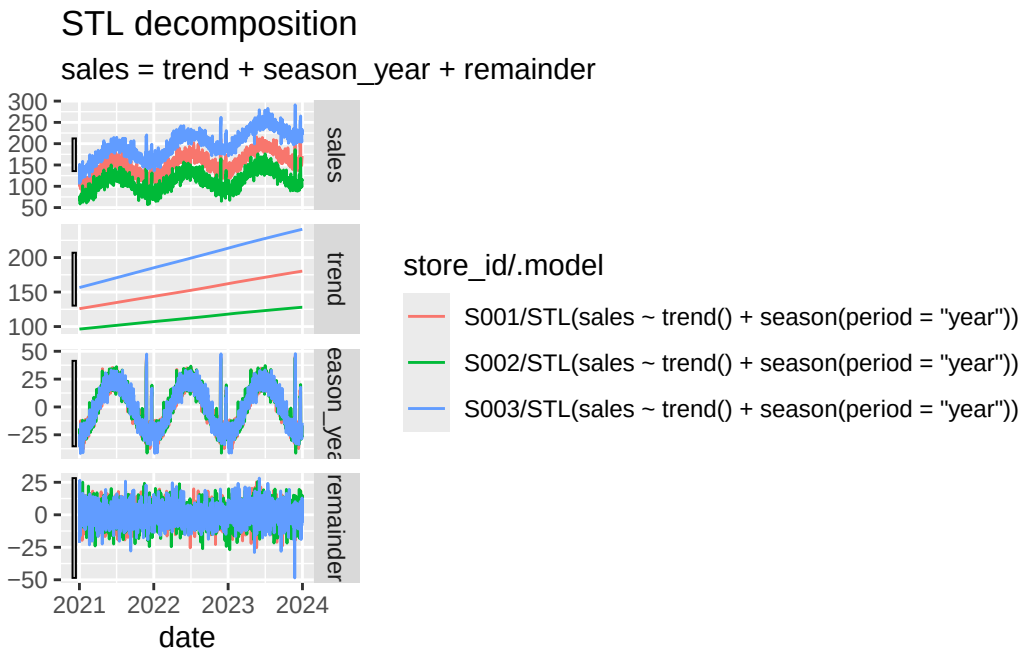

```
# Autocorrelation structure
sales_tsbl_filled %>% ACF(sales) %>% autoplot()
```

```
Warning: `autoplot.tbl_cf()` was deprecated in feasts 0.4.2.
i Please use `ggtime::autoplot.tbl_cf()` instead.
i Graphics functions have been moved to the {ggtime} package. Please use
  `library(ggtime)` instead.
```




```
# STL decomposition (needs an explicit period on daily data too)
sales_tsbl_filled %>%
  model(STL(sales ~ trend() + season(period = "year"))) %>%
  components() %>%
  autoplot()
```

```
Warning: `autoplot.dcmp_ts()` was deprecated in fabletools 0.6.0.
i Please use `ggtime::autoplot.dcmp_ts()` instead.
i Graphics functions have been moved to the {ggtime} package. Please use
  `library(ggtime)` instead.
```




```
sales_tsbl_filled %>%
  features(sales, feat_stl)
```

```
# A tibble: 3 x 10
  store_id trend_strength seasonal_strength_week seasonal_peak_week
  <chr>             <dbl>                  <dbl>              <dbl>
1 S001              0.904                  0.492                  2
2 S002              0.855                  0.497                  3
3 S003              0.942                  0.528                  2
# i 6 more variables: seasonal_trough_week <dbl>, spikiness <dbl>,
#   linearity <dbl>, curvature <dbl>, stl_e_acf1 <dbl>, stl_e_acf10 <dbl>
```

Reading and Diagnosing Data with `timetk`

```
library(timetk)
library(readr)

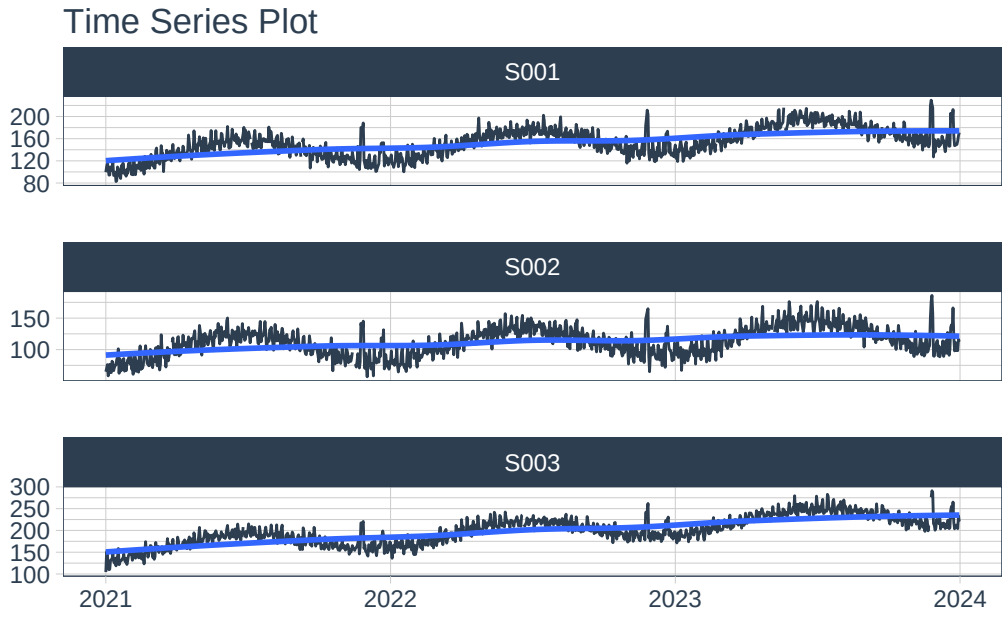
# Read data - timetk works with a normal tibble; only a Date column is required
sales_tbl <- read_csv(
  "sales_data.csv",
```

```
  col_types = cols(
    date     = col_date(format = "%Y-%m-%d"),
    store_id = col_character(),
    sales    = col_double()
  )
)

# Interactive/static time series plot, grouped by store (no tsibble conversion needed)
sales_tbl %>%
  group_by(store_id) %>%
  plot_time_series(date, sales, .interactive = FALSE)
```

```
Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.
i The deprecated feature was likely used in the timetk package.
  Please report the issue at
  <https://github.com/business-science/timetk/issues>.

Ignoring unknown labels:
* colour : "Legend"
```



```
# Diagnose the time index: regularity, gaps, scale, timezone
sales_tbl %>% tk_summary_diagnostics(date)
```

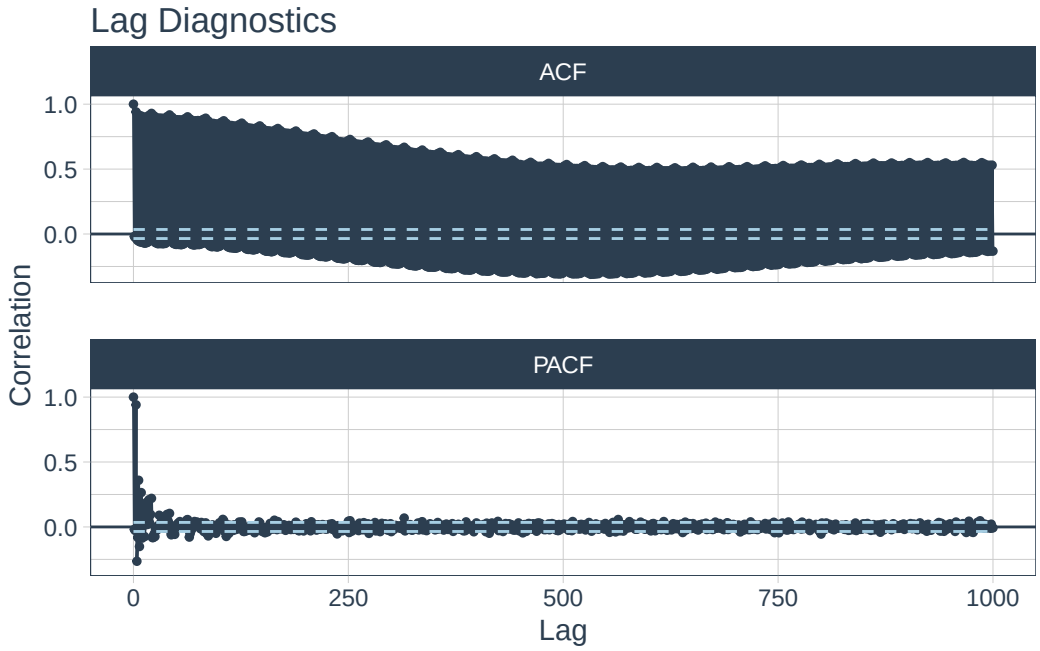
```
# A tibble: 1 x 12
  n.obs start      end        units scale tzone diff.minimum diff.q1 diff.median
  <int> <date>     <date>     <chr> <chr> <chr>        <dbl>   <dbl>       <dbl>
1  3285 2021-01-01 2023-12-31 secs  seco~ UTC              0       0           0
# i 3 more variables: diff.mean <dbl>, diff.q3 <dbl>, diff.maximum <dbl>
```

```
# Seasonal decomposition / ACF diagnostics in one call
# Explicit .feature_set: daily data has no sub-day granularity, so
# "second"/"minute"/"hour" are meaningless here and are excluded.
sales_tbl %>%
  plot_seasonal_diagnostics(
    date, sales,
    .feature_set = c("wday.lbl", "week", "month.lbl", "quarter", "year"),
    .interactive = FALSE
  )
```

```
Warning: Removed 75 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```



```
sales_tbl %>%
  plot_acf_diagnostics(date, sales, .interactive = FALSE)
```



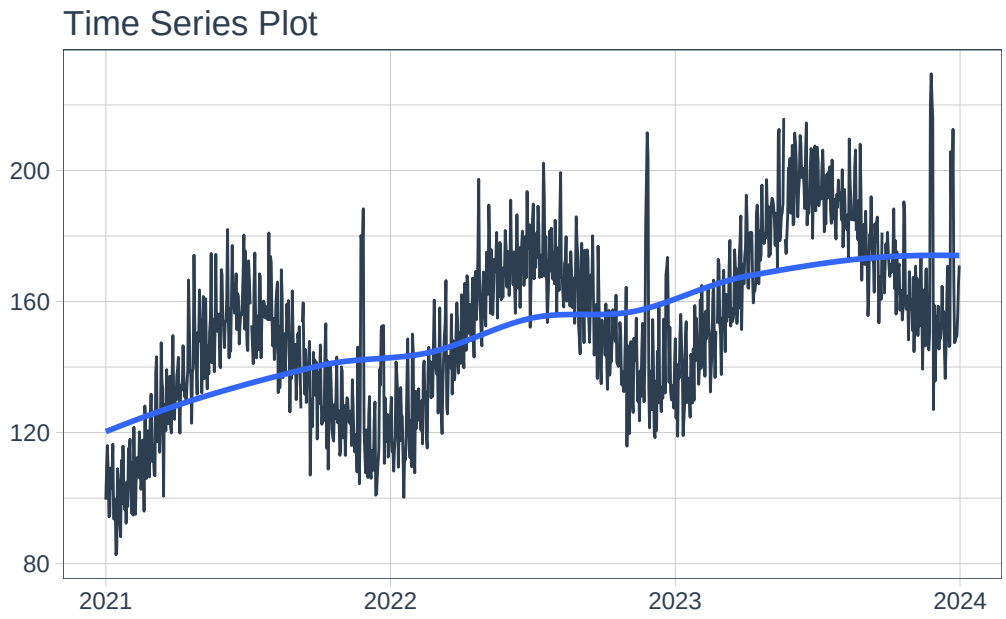
Plotting using all the different functions

```
library(timetk)

# Load and filter data
df <- read.csv("sales_data.csv")
df$date <- as.Date(df$date)
s1 <- subset(df, store_id == "S001")

# plot_time_series
s1 %>%
  plot_time_series(date, sales, .interactive = FALSE)
```

```
Ignoring unknown labels:
* colour : "Legend"
```




```
# tk_summary_diagnostics
s1 %>%
  tk_summary_diagnostics(date)
```

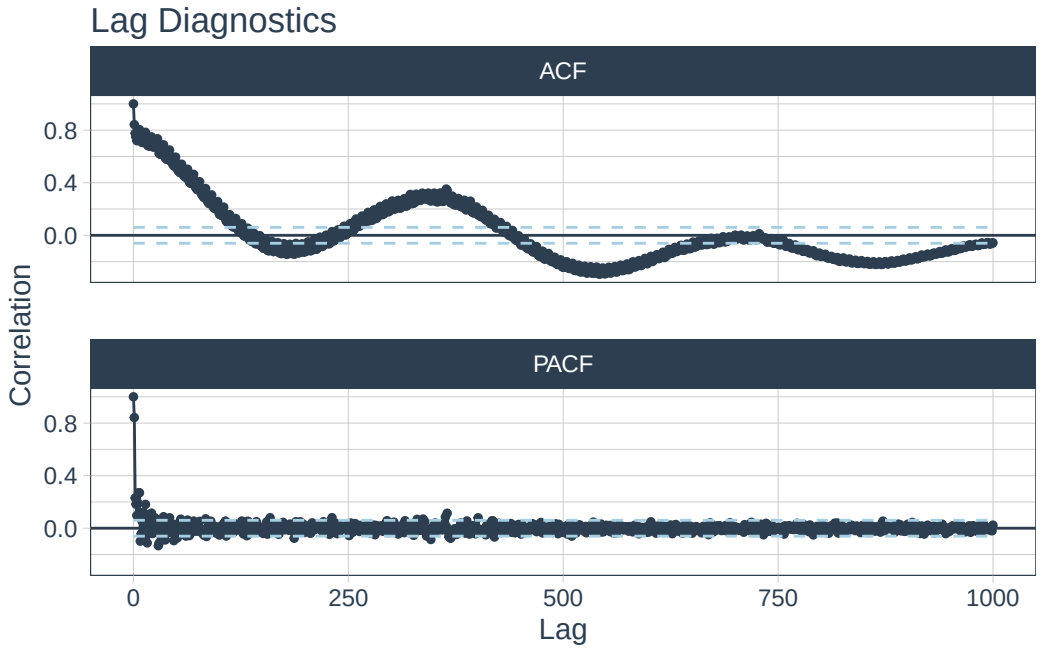
```
# A tibble: 1 x 12
  n.obs start      end        units scale tzone diff.minimum diff.q1 diff.median
  <int> <date>     <date>     <chr> <chr> <chr>        <dbl>   <dbl>       <dbl>
1  1095 2021-01-01 2023-12-31 days  day   UTC          86400   86400       86400
# i 3 more variables: diff.mean <dbl>, diff.q3 <dbl>, diff.maximum <dbl>
```

```
# plot_seasonal_diagnostics
s1 %>%
  plot_seasonal_diagnostics(date, sales, .interactive = FALSE)
```

```
Warning: Removed 25 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```




```
# plot_acf_diagnostics
s1 %>%
  plot_acf_diagnostics(date, sales, .interactive = FALSE)
```



```
# plot_anomaly_diagnostics
# s1 %>%
#   plot_anomaly_diagnostics(date, sales)
```

Question 7 - Answering code questions

Q1 — `ts(x, start = c(2023, 13), frequency = 12)`
This is invalid because monthly data only has 12 periods per year, so the second element of `start` can't exceed 12. Writing `c(2023, 13)` tries to say "period 13 of 2023," which doesn't exist — it should roll over to `c(2024, 1)` instead. Fix: `ts(x, start = c(2024, 1), frequency = 12)`.

Q2 — `ts(x, start = 2023, frequency = 12)`
For sub-annual data (anything with `frequency > 1`), `start` needs to specify both the year *and* which period within that year you're starting at — a single number is ambiguous. Fix: `ts(x, start = c(2023, 1), frequency = 12)`.

Q3 — `ts(x, start = c(2023, 0), frequency = 12)`
`ts()` uses 1-based indexing for periods, so period `0` doesn't exist within a year — the first month is period 1, not 0. Fix: `ts(x, start = c(2023, 1), frequency = 12)`.

Q4 — `ts(x, start = c(2023, 2), frequency = 4)`
The meaning of "period 2" depends entirely on what `frequency` you've set — with `frequency = 4` (quarterly), period 2 means Q2, not February. If the data is actually monthly, the frequency is wrong and needs to match the data's real periodicity. Fix: `ts(x, start = c(2023, 2), frequency = 12)`.

Q5 — `ts(x, start = c(2023, 1), frequency = 12.5)`
`frequency` must be a whole number, since it represents a count of periods per cycle (you can't have half an observation per year). Many time-series methods that build on `ts` objects require an integer frequency to work at all. Fix: `ts(x, start = c(2023, 1), frequency = 12)`.

Q6 — `ts(x, start = c(2023, 1), frequency = 12, deltat = 1/12)`
`frequency` and `deltat` both describe the same thing (the spacing between observations) from two different angles — one as "how many per year," the other as "time between = 1/frequency" — so specifying both is redundant and R doesn't allow it. Fix: pick one; here, `ts(x, start = c(2023, 1), frequency = 12)`.

Q7 — `ts(x, start = c(2023, 1), frequency = 365)`
`frequency = 365` tells R this is daily data, so if `x` is actually a vector of monthly values, the frequency doesn't match the actual data type/spacing — it would misassign what each value represents in the yearly cycle. Fix: `ts(x, start = c(2023, 1), frequency = 12)`.

Q8 — `window(x, start = c(2023, 13))`
`window()` can only extract a period that actually exists within the original series' time range —

asking it to start at "month 13 of 2023" (or any point outside the series' bounds) has nothing to return. Fix: pick a start point inside the series, e.g. `window(x, start = c(2024, 1))`.

Q9 — `decompose(ts(x, frequency = 1))`
`decompose()` splits a series into trend, seasonal, and random components — but with `frequency = 1`, there's no cycle at all (each "cycle" is a single point), so there's no seasonal pattern for it to extract, and it also needs at least two full cycles of data to estimate anything reliably. Fix: use a frequency that reflects real seasonality, e.g. `decompose(ts(x, frequency = 12))` for monthly data.